

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 1

Aim- Write a program for Merge Sort.

```
#include <iostream>

using namespace std;

void merge(int arr[], int l, int m, int r) {

    int n1 = m - l + 1;

    int n2 = r - m;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)

        L[i] = arr[l + i];

    for (int j = 0; j < n2; j++)

        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2) {

        if (L[i] <= R[j]) {

            arr[k] = L[i];

            i++;

        } else {

            arr[k] = R[j];

            j++;

        } k++;

    }

    while (i < n1) {

        arr[k] = L[i];

        i++;

        k++;

    }

}
```

Name- Yug Goud

Roll No.- 0822CS241228

```
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
    }

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    mergeSort(arr, 0, n - 1);
    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0; }
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of elements: 5

Enter elements:

45

12

67

23

9

Sorted array:

9 12 23 45 67

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 2

Aim- Write a program for Heap Sort.

```
#include <iostream>

using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
```

Name- Yug Goud

Roll No.- 0822CS241228

```
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);

        heapify(arr, i, 0);
    }
}

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    heapSort(arr, n);
    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of elements: 6

Enter elements:

30

10

50

24

69

40

Sorted array:

10 24 30 40 50 69

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 3

Aim- Write a program for Quick Sort.

```
#include <iostream>

using namespace std;

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
```

Name- Yug Goud

Roll No.- 0822CS241228

```
{  
    int pi = partition(arr, low, high);  
  
    quickSort(arr, low, pi - 1);  
    quickSort(arr, pi + 1, high);  
}  
}  
  
int main()  
{  
    int n;  
  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    int arr[n];  
    cout << "Enter elements:\n";  
    for (int i = 0; i < n; i++)  
        cin >> arr[i];  
    quickSort(arr, 0, n - 1);  
    cout << "Sorted array:\n";  
    for (int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
  
    return 0;
```


}

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of elements: 5

Enter elements:

25

17

32

67

2

Sorted array:

2 17 25 32 67

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 4

Aim- Write a program for Strassen's Matrix Multiplication.

```
#include <iostream>

using namespace std;

int main() {
    int n;

    cout << "Enter order of matrix (2x2 only): ";
    cin >> n;

    int A[2][2], B[2][2], C[2][2];

    cout << "Enter first matrix:\n";
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            cin >> A[i][j];

    cout << "Enter second matrix:\n";
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            cin >> B[i][j];

    int p1 = A[0][0] * (B[0][1] - B[1][1]);
    int p2 = (A[0][0] + A[0][1]) * B[1][1];
    int p3 = (A[1][0] + A[1][1]) * B[0][0];
```

Name- Yug Goud

Roll No.- 0822CS241228

```
int p4 = A[1][1] * (B[1][0] - B[0][0]);
```

```
int p5 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
```

```
int p6 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
```

```
int p7 = (A[0][0] - A[1][0]) * (B[0][0] + B[0][1]);
```

```
C[0][0] = p5 + p4 - p2 + p6;
```

```
C[0][1] = p1 + p2;
```

```
C[1][0] = p3 + p4;
```

```
C[1][1] = p1 + p5 - p3 - p7;
```

```
cout << "Result matrix:\n";
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
    for(int j=0;j<n;j++)
```

```
        cout << C[i][j] << " ";
```

```
    cout << endl;
```

```
}
```

```
return 0;
```

```
}
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter order of matrix (2x2 only): 2

Enter first matrix:

1 2

3 4

Enter second matrix:

5 6

7 8

Result matrix:

19 22

43 50

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 5

Aim- Write a program for the Greedy Knapsack Problem.

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Item {
    int profit, weight;
};

bool compare(Item a, Item b) {
    double r1 = (double)a.profit / a.weight;
    double r2 = (double)b.profit / b.weight;
    return r1 > r2;
}

int main() {
    int n;
    float capacity;
    cout << "Enter number of items: ";
    cin >> n;

    Item item[n];

    for (int i = 0; i < n; i++)
    {
```

Name- Yug Goud

Roll No.- 0822CS241228

```
cout << "Enter profit and weight of item " << i + 1 << ": ";
    cin >> item[i].profit >> item[i].weight;
}
cout << "Enter capacity of knapsack: ";
cin >> capacity;
sort(item, item + n, compare);

float totalProfit = 0;

for (int i = 0; i < n; i++)
{
    if (capacity >= item[i].weight)
    {
        capacity -= item[i].weight;
        totalProfit += item[i].profit;
    }
    else
    {
        totalProfit += item[i].profit * ((float)capacity / item[i].weight);
        break;
    }
}

cout << "Maximum profit = " << totalProfit;
return 0;
}
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of items: 3

Enter profit and weight of item 1: 60 10

Enter profit and weight of item 2: 100 20

Enter profit and weight of item 3: 120 30

Enter capacity of knapsack: 50

Maximum profit = 240

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 6

Aim- Write a program for the Huffman Coding.

```
#include <iostream>

#include <queue>

using namespace std;

struct Node
{
    char ch;
    int freq;
    Node *left, *right;

    Node(char c, int f)
    {
        ch = c;
        freq = f;
        left = right = NULL;
    }
};

struct compare
{
    bool operator()(Node* l, Node* r)
    {
        return l->freq > r->freq;
    }
}
```


Name- Yug Goud

Roll No.- 0822CS241228

```
};  
  
void print(Node* root, string str)  
{  
    if (!root)  
        return;  
  
    if (root->ch != '$')  
        cout << root->ch << " : " << str << endl;  
  
    print(root->left, str + "0");  
    print(root->right, str + "1");  
}  
  
int main()  
{  
    int n;  
  
    cout << "Enter number of characters: ";  
    cin >> n;  
  
    char ch[n];  
    int freq[n];  
  
    priority_queue<Node*, vector<Node*>, compare> pq;  
  
    for (int i = 0; i < n; i++) {
```

Name- Yug Goud

Roll No.- 0822CS241228

```
    cout << "Enter character and frequency: ";
    cin >> ch[i] >> freq[i];

    pq.push(new Node(ch[i], freq[i]));
}

while (pq.size() != 1)
{
    Node *left = pq.top();
    pq.pop();

    Node *right = pq.top();
    pq.pop();

    Node *top = new Node('$', left->freq + right->freq);

    top->left = left;
    top->right = right;

    pq.push(top);
}

cout << "Huffman Codes:\n";
print(pq.top(), "");

return 0;
}
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of characters: 4

Enter character and frequency: A 5

Enter character and frequency: B 9

Enter character and frequency: C 12

Enter character and frequency: D 13

Huffman Codes:

A : 00

B : 01

C : 10

D : 11

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 7

Aim- Write a program for the Optimal Merge Pattern using Greedy Method.

```
#include <iostream>
#include <queue>
using namespace std;

int main()
{
    priority_queue<int, vector<int>, greater<int>>> pq;

    int n, x;

    cout << "Enter number of files: ";
    cin >> n;

    cout << "Enter file sizes:\n";

    for (int i = 0; i < n; i++)
    {
        cin >> x;
        pq.push(x);
    }

    int cost = 0;

    while (pq.size() > 1)
```

Name- Yug Goud

Roll No.- 0822CS241228

```
{  
    int first = pq.top();  
    pq.pop();  
  
    int second = pq.top();  
    pq.pop();  
  
    int merge = first + second;  
  
    cost += merge;  
  
    pq.push(merge);  
}  
  
cout << "Optimal merge cost = " << cost;  
  
return 0;  
}
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of files: 4

Enter file sizes:

20

30

10

5

Optimal merge cost = 115

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 8

Aim- Write a program for Minimum Spanning Tree using Kruskal Algorithm.

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Edge {
    int src, dest, weight;
};

bool compare(Edge a, Edge b)
{
    return a.weight < b.weight;
}

int parent[100];

int find(int i)
{
    while(parent[i] != i)
        i = parent[i];

    return i;
}

void unionSet(int a, int b)
```

Name- Yug Goud

Roll No.- 0822CS241228

```
{
    parent[a] = b;
}

int main()
{
    int v, e;

    cout << "Enter number of vertices and edges: ";
    cin >> v >> e;

    Edge edge[e];

    for(int i=0;i<e;i++)
    {
        cout << "Enter source destination and weight: ";
        cin >> edge[i].src >> edge[i].dest >> edge[i].weight;
    }

    sort(edge, edge + e, compare);

    for(int i=0;i<v;i++)
        parent[i] = i;

    int minCost = 0;
```


Name- Yug Goud

Roll No.- 0822CS241228

```
cout << "Edges in MST:\n";

for(int i=0;i<e;i++)
{
    int a = find(edge[i].src);
    int b = find(edge[i].dest);

    if(a != b)
    {
        cout << edge[i].src << " - " << edge[i].dest
            << " : " << edge[i].weight << endl;

        minCost += edge[i].weight;

        unionSet(a,b);
    }
}

cout << "Minimum cost = " << minCost;

return 0;
}
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of vertices and edges: 4 5

Enter source destination and weight: 0 1 10

Enter source destination and weight: 0 2 6

Enter source destination and weight: 0 3 5

Enter source destination and weight: 1 3 15

Enter source destination and weight: 2 3 4

Edges in MST:

2 - 3 : 4

0 - 3 : 5

0 - 1 : 10

Minimum cost = 19

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 9

Aim- Write a program for Minimum Spanning Tree using Prim's Algorithm.

```
#include <iostream>

using namespace std;

int main() {
    int n;

    cout << "Enter number of vertices: ";
    cin >> n;

    int cost[10][10];

    cout << "Enter adjacency matrix:\n";

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            cin >> cost[i][j];

    int visited[10] = {0};

    visited[0] = 1;

    int ne = 1, mincost = 0;

    while(ne < n)
```

Name- Yug Goud

Roll No.- 0822CS241228

```
{  
    int min = 999, a = 0, b = 0;  
  
    for(int i=0;i<n;i++)  
    {  
        if(visited[i])  
        {  
            for(int j=0;j<n;j++)  
            {  
                if(!visited[j] && cost[i][j] < min)  
                {  
                    min = cost[i][j];  
                    a = i;  
                    b = j;  
                }  
            }  
        }  
    }  
}  
  
cout << a << " - " << b << " : " << min << endl;  
  
visited[b] = 1;  
  
mincost += min;  
  
ne++;
```

Name- Yug Goud

Roll No.- 0822CS241228

```
}  
  
cout << "Minimum cost = " << mincost;  
  
return 0;  
}
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of vertices: 4

Enter adjacency matrix:

999 10 6 5

10 999 999 15

6 999 999 4

5 15 4 999

0 - 3 : 5

3 - 2 : 4

0 - 1 : 10

Minimum cost = 19

Name- Yug Goud

Roll No.- 0822CS241228

Experiment No. 10

Aim- Write a program for Single Source Shortest Path Algorithm.

```
#include <iostream>

using namespace std;

int main(){
    int n, source;

    cout << "Enter number of vertices: ";
    cin >> n;

    int cost[10][10];

    cout << "Enter cost matrix:\n";

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            cin >> cost[i][j];

    cout << "Enter source vertex: ";
    cin >> source;

    int dist[10], visited[10] = {0};

    for(int i=0;i<n;i++)
        dist[i] = cost[source][i];
```

Name- Yug Goud

Roll No.- 0822CS241228

```
visited[source] = 1;
```

```
dist[source] = 0;
```

```
for(int count=1; count<n-1; count++)
```

```
{
```

```
    int mindist = 999, nextnode;
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        if(!visited[i] && dist[i] < mindist)
```

```
        {
```

```
            mindist = dist[i];
```

```
            nextnode = i;
```

```
        }
```

```
    }
```

```
visited[nextnode] = 1;
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
    if(!visited[i] &&
```

```
        mindist + cost[nextnode][i] < dist[i])
```

```
    {
```

```
        dist[i] = mindist + cost[nextnode][i];
```

```
    }
```

```
}
```


Name- Yug Goud

Roll No.- 0822CS241228

```
}
```

```
cout << "Shortest distances from source vertex:\n";
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
    cout << source << " -> " << i
```

```
        << " = " << dist[i] << endl;
```

```
}
```

```
return 0;
```

```
}
```

Name- Yug Goud

Roll No.- 0822CS241228

Output:

Enter number of vertices: 4

Enter cost matrix:

0 2 999 1

2 0 3 2

999 3 0 4

1 2 4 0

Enter source vertex: 0

Shortest distances from source vertex:

0 -> 0 = 0

0 -> 1 = 2

0 -> 2 = 5

0 -> 3 = 1